University of Inland Norway

Faculty of Audiovisual Media and Creative Technologies

Bachelor thesis

Saving the Game: Consistent Offline-Cloud State Synchronization in Survival Video Games

Lagre spillet:
Konsistent synkronisering av lokal- og skybasert tilstand i overlevelsesspill

B2VISIM Bachelor in Game Technology and Simulation

SPIS2900 Bachelor thesis

Hans Alan Whitburn Haugen

May 12, 2026

Abstract

Many survival games store their world entirely on remote servers. This means players cannot progress offline, and if the developer shuts down the online service, the game is lost forever. This thesis considers ways to design a system that supports both offline and online gameplay, and that allows a player to transition from an offline session to an online one without losing progress.

The main point is to log each player action in the form of a command instead of making the changes live on the server immediately. When the player is offline, the commands accumulate in a local queue. As soon as the connection is re-established, these commands are executed on the authoritative server in the same order. The biggest issue is how to reconcile conflicts: for example, if during the period of being offline, the building that a player intended to demolish was destroyed by another player, or the same territory was grabbed by another player, etc. then the system needs to figure out how to handle such situations. This thesis proposes a solution that divides commands into categories: there are commands that can always be executed irrespective of changes, some need a version check, and some still have to be rejected and communicated to the player.

A prototype was built in C++ to validate the design, including a SQLite-backed offline database and a durable command log that survives crashes.

Sammendrag

Mange overlevelsesspill lagrer spillverdenen utelukkende på eksterne servere. Dette gjør det umulig å spille spillet offline, og hvis utvikleren legger ned nettjeneste, så mister spillerne tilgang til spillet for alltid. Denne oppgaven undersøker hvordan et system kan utvikles som gjør at en spiller kan gå mellom en spillverden uten internett og en skybasert spillverden, uten å miste fremgang.

Hovedideen er å loggføre hver spillerhandling i form av en kommando, i stedet for å gjøre endringer direkte på serveren. Når spilleren er offline, samles disse kommandoene i en lokal kø. Så snart forbindelsen er gjenopprettet, spilles de av mot den autoritative serveren i samme rekkefølge. Den største utfordringen er å avgjøre hvordan konflikter skal håndteres: for eksempel, hva skjer når annen spiller har revet ned den samme bygningen eller inntatt det samme territoriet mens en spilte frakoblet? Denne bacherloroppgaven foreslår konflikthåndteringsstrategier som klassifiserer kommandoer etter type: det er kommandoer som alltid utføres uavhengig av hva som har endret seg, andre krever en versjonssjekk, og noen må rett og slett avvises og rapporteres tilbake til spilleren.

En prototype ble bygget i C++ for å validere designet av metoden, inkludert en SQLite-basert offline database og en kommandoliste som overlever at spillet plutselig lukkes.

Acknowledgements

I would like to thank my mentor, Tord Sindre Trøen, for helping me grow as a programmer and for giving me insight into the inner workings of professional game engine development and studio practices. His patience and guidance have strengthened my ability to think critically about code, particularly in the context of code reviews and the broader impact of technical changes. He also introduced me to effective ways of using AI to support code production, while maintaining a critical perspective on the generated results. In addition, he emphasized the importance of seeking help when needed and reaching out to others with prior experience in the systems I work with — advice that has reinforced similar guidance I have received from mentors in the past.

I would also like to thank Maria Esperanza Johnson Ruiz for her continued support throughout the course. I am grateful to Andreas Aasom Brandsnes, who has been an excellent collaborator and a kind friend on past projects. I would further like to thank Håvard Vibeto at the University of Inland Norway, Fredrik Martol at Funcom, and the Funcom Core Engine Team for giving me the opportunity to experience game development within a cutting-edge studio environment.

I would also like to thank my academic advisor at the University of Inland Norway, Ole Einar Flaten, who teaches 3D programming and game engine development—two of my primary areas of interest. His feedback and academic guidance have been invaluable throughout the thesis process, as well as during the internship period from 15 January to 13 May 2026.

AI has been used to help drafting prose, structuring arguments, and assisting with the implementation. Design decisions, benchmark data, and conclusions are my own.

Contents

Listings

1 Introduction

This investigation considers a way to develop the persistence system of a modern large-scale survival multiplayer game. The design targets persistent multiplayer worlds shared by thousands of players, and are intended to support the storage of player progression, world state, and faction dynamics. The design is not concerned with ephemeral information such as player position or real-time combat replication; the focus is on long-term persistent data of the kind typically stored in centralized, cloud-hosted SQL databases.

The suggested design is intended to provide a way to support both offline and online functionality. Having the ability to make a game run offline is important, especially on console platforms where access to online multiplayer features typically requires a paid subscription service. A significant portion of console players therefore do not participate in online multiplayer environments (Baker, 2025). Playing with friends should be supported via local cooperative modes, both splitscreen and local listen server setup, while preserving core gameplay mechanics.

This requirement introduces architectural challenges which need to be addressed sensibly. Games on the market which support persistent game state stored exclusively in cloud-based SQL databases assume continuous server connectivity. Supporting offline gameplay necessitates a hybrid architecture in which persistent world state can be stored locally, potentially using embedded database solutions such as SQLite (SQLite Consortium, 2024) which can later be synchronized with the authoritative cloud-based server infrastructure.

Many gamers prefer to play offline, but might want to connect online as they advance in the persistent world. Collaboration with other players is often necessary in survival games to navigate the harsh environment. Political dynamics and faction-based conflict play a central role in survival games, as groups of players compete for control and influence within the game's persistent world (Céré et al., 2021). Designs that allow users to transition online after initial offline sessions are explored.

The following research statement is investigated:

How can an offline-online synchronization system for game persistence targeting survival multiplayer games be designed and evaluated with respect to consistency, conflict resolution, and reconnection performance?

2 Methodology

A design science research approach (Hevner et al., 2004) has been adopted. In Design Science, the main product of the thesis is an artifact; in this project the artifacts are the architectural design of a save system and the actual implemented software prototype of the artifact. Instead of the empirical study, the software prototype is the main end product of the research.

The research is conducted in three phases. The first stage is analysis: it involves analyzing existing, server authoritative persistence systems and identifying the problems associated with playing offline. This analysis is performed in Section 4.

Proposal for second stage design: a hybrid architecture derived from the Command Pattern, with conflict resolution, and showing how the design fits together with online SQL backends and a local SQLite store, is proposed. Design decisions are recorded in Section 5.

The third stage is evaluation, implemented in C++, against two requirements:

1. Correctness, where all parts of the design are covered by automated unit and integration tests, written using the Catch2 framework (Hořeňovský & catchorg contributors, 2023).
2. Performance, where the replay throughput, command log size limits, conflict detection overhead, and the end-to-end cost of a full offline session are measured using a suite of benchmark programs.

Alternative solutions to the research question are briefly discussed in Section 6. The evaluation is described further in Section 7. The results are discussed in Section 8 and future work is touched upon in Section 9. Finally, a conclusion is made in Section 10.

3 Related Work

3.1 Game Industry Approaches

Several games available on the market have tried to solve the problem of syncing game data between playing offline and online. The game's developers do not typically disclose how they do it.

Valheim (Iron Gate Studio, 2021) uses SQLite to store its world state locally on player's machines. The entire saved game is found in `world.db`. SQLite makes the game fully playable offline. However, if two players host separate copies of the same world while disconnected, there is no supported way to rejoin and merge the database differences. This is the snapshot problem described in Section 4. SQLite makes it straightforward to implement single-player sessions but rejoining an online session with SQLite introduces challenges regarding cheating and consistency.

Steam Cloud Save (Valve Corporation, 2023) enables folders and files to be synchronized between computers by hosting them on Valve's servers. The save data is uploaded and downloaded when players connect to Steam. Conflicts are resolved by making the most recently modified file the one which is stored on the server, a last-write-wins system. This approach is simple and adequate for single-player games, but cannot handle merging modifications from multiple players.

World of Warcraft (Blizzard Entertainment, 2004) is one of the earliest examples of a MMO which is based on the server-authoritative model. All of the persistant game state is stored on Blizzard's servers. All items, character properties, quests and guild information is stored remotely. When Blizzard stopped the legacy vanilla servers in 2016, all the game data, the characters, player progression, everything done in the game world was lost, which is another example of the preservation problem discussed in Section 8.

Blizzard Entertainment's Diablo III (Blizzard Entertainment, 2012) required online connectivity to the Battle.net service even for solo gameplay, their new game required all character state to be stored and validated server-side. At launch in 2012 server overload made the game completely unplayable despite no multiplayer interaction being involved. This is widely referred to as "Error 37" and illustrates how the purely server-authoritative model can fail. If the game depends entirely on the availability of the developer's infrastructure, regardless of whether the player is interacting with others, the game becomes unplayable if the developers have server problems.

Many large-scale survival multiplayer games, such as Rust and DayZ, take the same approach. Offline play becomes impossible, requiring a constant internet connection to a central server. This sidesteps offline to online persistence synchronization entirely, but at the cost of offline-mode.

Academic distributed systems literature has studied the problem and offers so-

lutions, though they are often overly complex:

Conflict-free Replicated Data Types (CRDTs) (Shapiro et al., 2011) are data structures that can be replicated over several computers and can be changed immediately with a guarantee of eventual consistency with the other computers. Any two replicas can be merged out of order but will eventually give the same result on all computers. CRDTs would eliminate the need for conflict resolution policies entirely. However, the entire game would need to be redesigned. Every data type would have to adhere to CRDT semantics. This is a significant architectural investment. They are also better suited to data types like counters and sets than to the complex relational structures (buildings with positions, quest state, faction information, map markers) that survival games require.

Operational Transformation (OT) (Ellis & Gibbs, 1989), the technique underlying WYSIWfS (what you see is what I see) editing tools such as Google Docs, transforms operations so that they can be applied out of order and still produce the same document. The intent-based approach is similar to the model in this thesis, commandlog shares similarities with OT. The command-based approach and OT both consider operations rather than snapshots. However, OT requires a central server to sequence and transform operations in real time. This is not possible during an offline play session.

Event sourcing (Fowler, 2005) is a software architecture pattern where the program state is constructed by unwinding an append-only log of events. The command log proposed in this thesis is very similar to this architecture: both record intent rather than state and both allow the current state to be reconstructed by replaying the log. The main difference is that event sourcing assumes a single authoritative log, whereas this thesis addresses what happens when two divergent logs (client offline, server online) must be merged.

The conflict resolution strategy, which uses entity versioning and optimistic concurrency control, follows the approach described by Bernstein and Goodman (Bernstein & Goodman, 1981). Here, transactions validate the legality of actions at commit time rather than acquiring locks upfront. It is typical to make data consistent by relying on mutexes: a thread or process is given an exclusive lock before modifying shared state, blocking all other writes until the lock is released. This works well when everyone is continuously connected to an online server, but for offline sessions holding a lock while disconnected would stall the entire system. Optimistic concurrency control solves this by allowing offline progress to proceed freely and doing conflict detection when users reconnect to the server. This is why the conflict resolution from Bernstein and Goodman becomes a natural fit for the Command-based offline–online model presented in this thesis.

Kleppmann et al. (Kleppmann et al., 2019) calls software which should work offline a *local-first* software problem: applications should keep data locally, not on the cloud, and remain fully functional without a network connection. Synchronization with collaborators or cloud services should only occur when the

user is ready and connects online. The command log approach proposed in this thesis is a *local-first* problem designed to be compatible with survival multiplayer games. The commandlog operates on persistent state that is shared by many players. In the model presented in this thesis, an authoritative server must eventually become the final source of truth.

Moll et al. (Moll et al., 2021) present a quadtree-based protocol to sync game state across server clusters, so called Named Data Networking. While they focus on inter-server sync rather than offline play, they show that region-based naming and decoupled representations can cut the overhead of keeping distributed states consistent. A concern related to the scalability dimension of the problem addressed later in the thesis in Section 8.3.

4 Typical Challenges in Persistence Systems

Game worlds consist of actors that may own inventory items, buildings, vehicles, navigation markers, abilities, and quest progression. In large-scale online survival games, persistent game state is commonly stored in centralized persistence systems backed by SQL databases hosted on cloud infrastructure.

Such systems typically follow a server-authoritative architecture in which all state-modifying actions are processed through a centralized backend. Player actions are buffered temporarily on the client before being committed to the authoritative server. Multiple server instances may run concurrently, but at any given time only one instance holds authoritative control over a given player or region. Actions are executed as database transactions, providing rollback in the event of failure.

A common design pattern for this kind of system is to apply the Service Locator pattern (Nystrom, 2014b) to abstract the communication layer responsible for generating and dispatching persistence commands. This abstraction is hereafter referred to as the Database Interface (DBI). Rather than issuing raw SQL queries directly from game code, the DBI invokes predefined server-side functions. Each function executes as a transactional unit and can be rolled back if any step fails, so every state-modifying action is encapsulated as an atomic database operation.

A well-designed DBI supports atomicity, consistency, and idempotency. If a network failure occurs, or if the client is uncertain whether a transaction completed, the DBI can safely retry without risking duplicate or inconsistent state changes. An important structural advantage of this abstraction is that the DBI can be swapped out via the Service Locator pattern: an online session routes calls to the remote SQL backend, while an offline session could route them to a local implementation — the implications of which are explored in Section 5.

A SQL-based DBI architecture treats the cloud-hosted SQL database as the single authoritative source of truth for all persistent game state. This provides strong consistency guarantees in fully online environments, but it inherently depends on centralized authority and does not account for state divergence in disconnected scenarios. See the listing below:

Listing 1: DBI interface (abstract base class)

```
class IDatabaseInterface
{
public:
    virtual ~IDatabaseInterface() = default;

    virtual bool CreateBuilding(int32_t playerId, int32_t buildingTypeId,
                                float posX, float posY, float posZ) = 0;
    virtual bool DestroyBuilding(int32_t buildingId) = 0;

    virtual bool TransferItem(int32_t fromPlayerId, int32_t toPlayerId,
                              int32_t itemId, int32_t quantity) = 0;

    virtual bool CompleteQuest(int32_t playerId, int32_t questId) = 0;

    virtual bool UpdateFactionRelation(int32_t factionA, int32_t factionB,
                                       int32_t relationDelta) = 0;
};
```

4.1 Limitations of Server-Authoritative Architectures

A server-authoritative persistence architecture requires continuous internet connectivity to a centralized cloud-based server. Although this model is good for consistency, it entails significant drawbacks when developers want to support offline or hybrid offline-online game modes:

Since all actions that will change the database data are executed as remote transactions, gameplay cannot progress without server access. This makes offline play impossible. If the online service is shut down, the game will no longer function. It also becomes more difficult to deploy the game on platforms where online access requires a paid subscription. If a player loses internet connectivity mid-session, for example due to server maintenance, gameplay cannot continue. The client has no way to queue actions for later submission. For players from regions with unreliable internet connectivity, the game becomes more or less inaccessible. The combined architecture described in Section 5 tackles these limitations by decoupling action generation from action transmission.

In a purely server-authoritative model, the client does not have a local database of the world state. While a temporary state may exist in memory during a session before being transmitted to an online server, durable storage is managed exclusively by remote infrastructure. This prevents local world progression during disconnection and support for local cooperative play modes becomes difficult to implement. Implementing a local persistence layer, such as the SQLite-backed DBI described in Section 6.1, enables progress to persist across by writing state to disk which will allow offline sessions.

A server-authoritative architecture expects all state changes to happen linearly and does not allow modifications from offline sessions. For example, both offline players and the live server may modify the same entity independently. As a result, such architectures usually lack:

1. **Entity version tracking**: Without having a way to know which version of an entity a player has acted on, it is impossible to determine whether the action has happened on outdated information.
2. **Conflict identification**: Without entity version tracking it is not possible to detect conflicts which occur when two or more players modify the same entity in conflicting ways.
3. **Deterministic merging**: When a conflict has been detected, conflict resolution must take place. An authorative server typically has no rules for such an action when all actions happen in transactions: last-write-wins, action rejection, or type-specific merging can be implemented to support offline sessions.

Without the above points, synchronization of locally modified state with cloud-based state is not possible. The version-based conflict recognition and command classification strategy described in Section 5.3 addresses all three gaps.

5 Proposed Hybrid Persistence Architecture

5.1 Command-Based State Replication

To enable offline progression without completely redesigning the server-authoritative model, this thesis proposes a command-based persistence model inspired by the Command Pattern as originally described in Design Patterns: Elements of Reusable Object-Oriented Software (Gamma et al., 1994) and presented specifically for games development in Game Programming Patterns (Nystrom, 2014a).

Rather than directly transmitting changes to the cloud-based database, during offline sessions the player actions are encapsulated as serializable command objects. Each command represents an intention instead of just a state change. An action is for example to create a building, transfer an item, complete a quest or place map marker. A command will also have parameters which allows the server to convert the command into a SQL modification which can be executed on the server when the player reconnects online.

In other words, during an offline session commands are appended to a local command log. The commands are not transmitted immediately to the server. When connectivity is restored, the queued commands are transmitted sequentially to the authoritative backend cloud-server and executed as transactional operations.

This approach introduces several advantages:

1. Commands are easy to serializable and can be stored locally.
2. Replaying commands enables deterministic reconstruction of state.
3. The command log provides a history of offline actions.
4. The server-side transaction logic does not need to change at all.
5. Fraud detection can rely on the old system.

By storing intent rather than state, the game will not diff to databases and merge them but instead achieves synchronization by executing state-modifying operations in order. See the listing below:

Listing 2: Command structure and local command log

```
struct Command
{
    std::string type;             // e.g. "CreateBuilding", "AddResource"
    int64_t     timestamp;        // UTC milliseconds at time of action
    std::string idempotencyKey;   // UUID to detect duplicate replays
    std::string entityId;         // ID of the affected entity
    int32_t     expectedVersion;  // entity version at command creation
    int32_t     delta;            // delta for commutative operations
};

class CommandLog
{
public:
    void                    Append(const Command& cmd);
    std::vector<Command>    GetPending() const;
    void                    MarkFlushed(const std::string& idempotencyKey);
    void                    MarkFlushedBatch(
                                const std::vector<std::string>& keys);
    std::size_t             PendingCount() const;

private:
    std::vector<Command> m_pending;
};
```

5.2 Implementation considerations

Security considerations are important when dealing with online games, it is important to make sure players do not cheat and create false information. This system must be designed to be secure. The suggested system allows commands to be generated client-side and stored locally. A malicious client could potentially create or alter commands and add them to the local queue before transmission to the server. Every command must therefore be validated server-side before a transaction is committed, and any command that produces an invalid or impossible state transition must be rejected. Conflict detection, entity versioning, and per-type resolution policies are addressed in the following subsection.

Two other concerns need to be addressed as well. The first one is ordering guarantees: the command log must always represent a sequence of commands in in the order they were taken while offline. The server must replay them in that order to ensure deterministic results. Second, idempotency. If the same command is added twice, which could happen due to bugs in the gameplay code or unexpected player behaviour, or if the execution of a command fails, the server must be able recover and skip said commands. The idempotency key found in the command-struct (see listing above) serves this purpose.

Lastly, what should the server do if commands depend on entities which have been removed on the server-side? If an offline command expects an entity to exist which has been deleted by another player during an online session, the commands will no longer make sense. Handling this requires more than simple version-checks, the game will need to be able to provide a rejection manifest of items or structures which could not be applied to the online database due to divergence in the online game state of the game world.

5.3 Conflict Resolution Strategy

When offline commands are replayed against the authoritative server, conflicts can arise. When the server state diverges from the state the offline player

world state, the commands generated might need to be rejected. As the game world evolves and diverges online, commands generated during offline sessions could become invalid or inconsistent with the state on the server and will not transition to the online server during a reconnection online. Ways to handle this are explored next:

5.3.1 Conflict Detection via Entity Versioning

Each persistent entity in the database (inventory item slot, map marker, quest, building, etc.) is assigned an ever-increasing version number. This is called a monotonically incremented number which increases with every committed state change. This is the expectedVersion field in the listing above. When a command is generated offline, it will look into the local database for the version of the entity the player has manipulated and add it to the command. When the player reconnects and the commands are replayed, the server will compare the recorded version against the current entity version. The online version number will increase and go out of sync with the offline version number if other players have manipulated the same object, and conflict resolution can take place, for example the command can be rejected.

This is very similar to Optimistic Concurrency Control (OCC) which is used in traditional database systems. Transactions proceed without locking but are validated at commit time (Bernstein & Goodman, 1981).

5.3.2 Command Classification

Just because entity versions do not match between local and server sessions, the command does not necessarily need to be rejected. Commands have different types and different conflict resolution strategies:

1. **Idempotent commands** — Quest completion, achievement unlocks, and similar boolean flags. These produce the same result regardless of how many times they are applied and can be safely replayed without harm.
2. **Commutative commands** — Additive operations such as resource collection, experience points, etc. The order does not change the final value. These commands can simply have their deltas summed together with the existing entity.
3. **Non-commutative commands** — Spatial operations or building related such as destroying a building or placement of a vehicle. These need to be rejected and communicated to the reconnecting player if there is a conflict with server entities.

5.3.3 Resolution Policies

For each conflict type a deterministic resolution must take place on the server before the command is committed. See listing below:

Listing 3: Server-side command validation and conflict resolution

```
ReplayResult ReplayCommand(const Command& cmd, Database& db,
                           RejectionManifest& manifest)
{
    // Idempotent: already applied, skip silently
    if (db.IsAlreadyApplied(cmd.idempotencyKey))
        return ReplayResult::Duplicate;

    // Commutative: apply delta regardless of entity version
    if (cmd.type == "ResourceDelta" || "ReputationDelta")
        return db.ApplyDelta(cmd);

    // Non-commutative: validate entity state before committing
    Entity entity = db.GetEntity(cmd.entityId);

    if (!entity.exists)
    {
        manifest.push_back({cmd, ReplayResult::Rejected});
        return ReplayResult::Rejected;
    }

    if (entity.version != cmd.expectedVersion)
    {
        manifest.push_back({cmd, ReplayResult::Conflict});
        return ReplayResult::Conflict;
    }

    return db.Commit(cmd);
}
```

5.3.4 Rejection Manifest

Commands that fail conflict resolution can not be silently discarded. They need to be collected and displayed to the player. This is the rejection manifest, a warning about what will not be applied to the game state if the player chooses to reconnect and what will be done about the changes. The manifest should show the command which was rejected, why it was rejected, and what on the server stopped it from being applied. This way the game has a way to warn the player about the consequences for reconnecting online, what will happen if they do so, and give the player a way to understand what happened during reconnection instead of simply finding the world in a different state than expected. Items or buildings which could not be committed could be stored in a server-side backpack and accessed by the player via a menu in the game.

5.4 Command Dispatch and SQL Execution

Once `ReplayCommand` has processed a command and found it can be applied to the game state, it calls `db.Commit(cmd)`. This will check the `type` field of the Command and a DBI method is called. The Command is translated into SQL statements which can be applied on the server. Each command type corresponds directly to a single DBI method which produces SQL code and a transaction on the server. See the listing and table below:

Listing 4: Server-side command dispatch to DBI methods

```
ReplayResult Database::Commit(const Command& cmd)
{
    if (cmd.type == "CreateBuilding")
        dbi.CreateBuilding(cmd.entityId, cmd.delta, 0, 0, 0)
            ;
    else if (cmd.type == "DestroyBuilding")
        dbi.DestroyBuilding(cmd.entityId);
    else if (cmd.type == "TransferItem")
        dbi.TransferItem(cmd.entityId, cmd.delta);
    else if (cmd.type == "CompleteQuest")
        dbi.CompleteQuest(cmd.entityId, cmd.delta);
    else if (cmd.type == "ResourceDelta" ||
             cmd.type == "ReputationDelta")
        dbi.UpdateFactionRelation(cmd.entityId, cmd.delta);

    RecordApplied(cmd.idempotencyKey);
    IncrementVersion(cmd.entityId);
    return ReplayResult::Committed;
}
```

Each DBI method executes SQL with the properties from the command:

Command type	DBI method	SQL pattern
`CompleteQuest`	`CompleteQuest`	`INSERT OR IGNORE INTO completed_quests`
`TransferItem`	`TransferItem`	`BEGIN; SELECT / UPDATE / UPSERT; COMMIT`
`CreateBuilding`	`CreateBuilding`	`INSERT INTO buildings`
`DestroyBuilding`	`DestroyBuilding`	`DELETE FROM buildings WHERE id = ?`
`ResourceDelta`	`UpdateResource`	`INSERT ... ON CONFLICT DO UPDATE SET`
`ReputationDelta`	`UpdateFactionRelation`	`relation = relation + excluded.relation`

The above are some examples of SQL patterns which could be used for the commands. `INSERT OR IGNORE` is for idempotent commands, these can be called many times without changing the database state. With this duplicates from for example calling a Quest command many times, will be ignored. This is really just an extra fail-safe in-case the built-in idempotency detection in the ReplayCommand method fails. (`ON CONFLICT DO UPDATE SET value = value + excluded.value`) is for resource and reputation deltas, it simply adds the resource or reputation delta to the column. `BEGIN/COMMIT` is a transaction for `TransferItem`. Transactions like these ensures atomicity: either both succeed or neither does. This prevents an inventory item from being lost or getting duplicated in-case the transaction fails. For existing games one might already have a DBI implementation, each command must simply correspond to a DBI method.

5.5 Integration Possibilities

In a server-authoritative game, every action which has to do with persistence should happen through a common Database Interface (DBI). The DBI could become a factory that produces command objects, and depending on the player session the DBI could be swapped out: online commands would use a DBI where actions take place on the server immediately, while the offline DBI would queue up commands and store them on disk. When the player reconnects online, the queue is flushed and conflicts dealt with.

The offline DBI would still have a database implementation, implemented in for example SQLite, to allow the game to know about its own state. When a player opens the inventory in the game, for example, the game needs to have a way to ask a database what is in the inventory. So the offline DBI would consist of a SQLite database, for world state, and a queue to store the intent of the player for later synchronization with the online authoritative server. Ideally the SQL commands would be identical on the online server and on the offline SQL database, simplifying implementation.

The command routing is straightforward to implement. A DBI factory checks connectivity and returns the appropriate DBI implementation:

Listing 5: Connectivity-aware DBI factory

```
std::unique_ptr<IDatabaseInterface> CreateDBI(bool isOnline)
{
    if (isOnline)
        return std::make_unique<RemoteSQLDBI>(serverConfig);
    else
        return std::make_unique<SQLiteDBI>("offline.db");
}
```

Game code calls DBI methods without concern of which backend has been created. The DBI has a common interface and can be swapped out. When the player rejoins an online session, the engine flushes command log by calling `ReplayAll`, this submits all the commands from the command queue to the authoritative server in the order they happened in the offline session and conflicts produce a `RejectionManifest` listing any commands that could not be applied. This list could be made accessible to the player after reconnection and a method to recover lost items or buildings could be implemented.

Figure 1 shows a flow chart of how the DBI system:

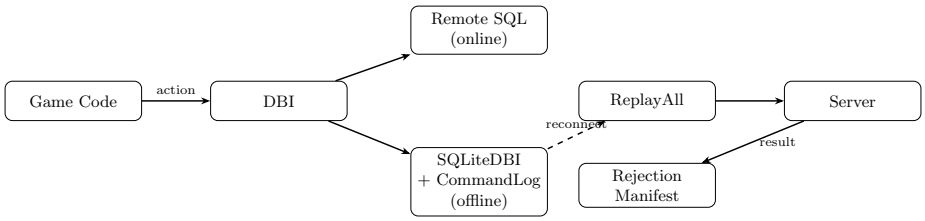

Figure 1: Hybrid persistence architecture: game code interacts only with the DBI; the active implementation is swapped at runtime depending on connectivity.

5.6 Prototype Implementation

A self-contained C++ prototype of the proposed architecture was developed alongside this thesis and is available at:

https://github.com/alanhaugen/SPIS2900-Bachelor

The prototype is located in the `app/` directory. The prototype libraries and benchmark program is built using CMake. To ensure correctness of the implementation, the testing framework Catch2 was used. CMake has been configured to fetch the library automatically at configure time. A system SQLite3 library is expected. On MacOS, SQLite3 is part of the base system and is already installed. The implementation is organized as two libraries:

`offline_sync_lib` contains the core persistence abstractions without a SQLite database:

- `IDatabaseInterface` — this is the abstract DBI used by game code, it is a factory following the Service Locator pattern (Nystrom, 2014b), with purely virtual methods for all persistent game actions.
- `Command` and `CommandLog` — an in-memory implementation of command queue described in Section 5.1.
- `Database` — an alternative lightweight in-memory entity store, used for testing, with version tracking and idempotency key recording.
- `ReplayCommand` / `ReplayAll` — the conflict-resolution algorithm from Section 5.3, including the `RejectionManifest` type.

`offline_sync_sqlite_lib` contains the SQLite-backed implementations as suggested in the thesis:

- `SQLiteDBI` — a concrete implementation of `IDatabaseInterface` that stores game state in a local SQLite file. It maintains game-domain tables for buildings, item inventories, completed quests, and faction relations. Item transfers execute inside an explicit SQLite transaction and are rejected atomically if the sender has insufficient inventory. Quest completion uses `INSERT OR IGNORE` to preserve idempotency. Faction relation updates use an UPSERT to accumulate signed deltas.
- `SQLiteCommandLog` — a durable replacement for the in-memory `CommandLog` that writes each command to a SQLite file before returning. This is one way to store the commandlog. The `idempotency_key` column carries a `UNIQUE` constraint so that a double-append is silently discarded at the database level. Commands survive process restarts and can be recovered on next open.

The `offline_sync_lib` is simpler and does not support commandlog survival and has a simpler local database implementation, while the `offline_sync_sqlite_lib` implementation uses SQLite for the local game world state and another SQLite database to store the commandlog. The two implementations demonstrate how the `IDatabaseInterface` abstraction allows the implementation of the DBI to be swapped out without changing game code.

The prototype includes 31 Catch2 unit tests covering the command log, idempotency, commutative merging, non-commutative conflict detection, SQLite persistence, and an offline-session scenario.

6 Alternative State Replication Persistence Architectures

Other simpler alternatives to the Command-Based State Replication have been considered and will be described and discussed below. Some of the problems with the alternative architectures revolve around not allowing players to reconnect online after an offline session due to state divergence (see section 6.1) and the added complexity of the implementation in addition to not solving the problem of allowing players reconnect online (see section 6.2).

6.1 Local Persistence via SQLite

A most obvious way to add offline functionality to a game running the server-authoritative model is to implement an offline Database Interface (DBI) implementation with a complete port of the implementation to SQLite. If the cloud-server is already using SQLite, this process becomes even simpler. Rather than sending commands to remote SQL servers, the DBI would simply execute equivalent commands on a local SQLite database during an offline session.

SQLite (SQLite Consortium, 2024) is an embedded, serverless, ACID-compliant relational database engine. It can be included into any C++ program as a simple and lightweight static library. This makes this implementation easy to support, other SQL implementations usually require a separate server process running in the background. This is not supported on all platforms, such as console. SQLite is embedded inside the main application and stores data in a single file on disk. This makes it well-suited for offline persistence on both personal computers and consoles.

By using a DBI abstraction layer, the game can have a consistent programming interface for persistence operations which can stay the same for both offline and online sessions. Gameplay code does not need to be touched or rewritten. When offline, all DBI calls would be routed to the SQLite backend. When online, all DBI calls would sent remotely to the online SQL infrastructure.

Such a design would result in minimal changes to the game code, the persistence boundary remains unchanged. However, how do you deal with players who want to return to an online session? How do you copy an online database down to a private offline session? If the online server uses a different SQL implementation than the offline SQLite schema, conversion wrappers must be written. Transaction semantics might need to be reconsidered, synchronization between the local SQLite instance and the authoritative cloud database can become very complicated and detecting fraud would be incredibly difficult.

A SQLite-backed DBI alone is not enough to enable offline-online synchronization. The SQLite DBI is a *readable state store*, it answers the question "what does the world look like right now?" and allows the game to display inventory, buildings, and quest progress during an offline session. However, it does not

have information about what has *changed* during the offline session. On reconnection, the server will not have any way to know which operations the player has actually performed, all it gets is a complete snapshot of the game state with information on how the game has diverged from the online game. Merging two divergent snapshots is complicated, it will require diffing the two databases, find every entity affected, and apply the changes. This might not even be possible as the divergence becomes greater during a play session, and entities are deleted and added on both the online and offline session.

A structural diff is insufficient for two reasons. First, a diff records *what changed*, not *what the player did*: if a player's money balance fell by 20, the diff sees -20. However, what the player did becomes invisible. Was the change due to a purchase, a discovered treasure chest, or a quest being completed? With a snapshot diff only, there is no way for the server to determine how the diff happened and server-side validation rules become impossible to implement. Without knowing the original action, the server cannot classify the change or apply a meaningful resolution policy. Second, operations that cancel out during an offline session leave no trace: if a player builds a structure and then demolishes it while offline, it would be the same as doing nothing in the game to the online server. Yet those actions will consume resources and trigger side-effects such as experience points gained which should be validated on the server. A command log captures every step while a diff only captures the outcome.

This is why the command log is a necessary complement rather than an optional addition. The command log is a *pending change queue*. It is a record of all the actions that the player has done in the order the actions were made, it shows the player intent and can be safely replayed on the server. After a successful replay the log is flushed and forgotten about. Then the SQLite database can be forgotten as well and the player can return to an online SQL database. The two components answer different questions. One cannot substitute the other: removing the command log leaves no replay record, while removing the SQLite DBI leaves the game unable to read any state, such as what the inventory currently holds, or what quests are active, while offline.

6.2 ORM (Object-Relational Mapping)

At the beginning of the project, an Object–Relational Mapping (ORM) system was considered for the project. An ORM is a reflection layer which allows objects to translate directly to SQL and back again. This is a technically interesting solution which would allow all persistent state (e.g., inventory items, buildings, quests, etc.) to automatically get translated into the database as they are manipulated on the client and to updated and reconstructed from relational data as well.

ORM frameworks are not typical for C and C++ but are often seen in higher-level languages such as Python and Java. Languages with native reflection and dynamic typing make it much easier to implement runtime type inspection

and to create SQL schema based on objects in memory. C++ does not have reflection and is statically typed, which makes it complicated to make a program to examine itself at runtime and add SQL serialization logic. However, a game engine like Unreal engine extends C++ with a reflection system, each object has extra information thanks to macros such as `USTRUCT` and `UCLASS` (Noland, 2014). This system could be extended to support an ORM.

The deeper problem, however, is a that the ORM model still does not solve synchronizing offline-online state. The command-based architecture proposed in this thesis is still the only solution presented which actually takes care of this problem. An ORM operates on *state*: just like the SQLite-only implementation, it can only track which object fields have changed and generate SQL based on snapshots. The command log approach considers *operations*: it logs player actions and intent, not the resulting world state only. These two models are philosophically opposed. If an ORM were used as the persistence layer, it would not generate a record of actions and would lose the intent that the command log gathers up for server-side replay. With ORM the server would only receive a snapshot of what changed, it would not be able to replay the actions in order and deal with conflict resolution in a straightforward manner. That might even be impossible with ORM.

The DBI abstraction is already an excellent backend-agnostic abstraction, replacing it with ORM would simply add complexity. Game code should interact with a interface and should not be aware of the backend-implementation. The game code should not care if the game is running online, or if it is being used in an offline session. A great thing about the DBI is that is exposes all persistence methods in one place, such as (`TransferItem`, `CompleteQuest`), rather than having generic persistence in different places across the codebase. Without this, it becomes difficult to classify commands for conflict resolution. The DBI is a domain-specific and explicit, it is better suited than a generic ORM. A DBI is the best solution for a persistence system due to its simplicity and compatibility with existing authoritative server architectures.

7 Evaluation

The prototype was evaluated along two dimensions: correctness, verified through automated testing, and performance, measured through a benchmark suite run on the same hardware used for development (Apple M-series, macOS, Release build with compiler optimisations enabled).

7.1 Correctness

The implementation is covered by 31 Catch2 unit and integration tests organised into five categories: command log behaviour (append, ordering, flush), idempotency (duplicate commands produce no side effects), commutative merging (resource and reputation deltas apply regardless of version mismatch), non-commutative conflict detection (version mismatch and missing entities are caught and reported), and a full offline session integration test that exercises all result types in a single replay pass. All 31 tests pass.

7.2 Replay Throughput

The first benchmark measures how quickly the in-memory replay engine can process a command log of increasing size, using one unique entity per command to isolate the cost of the replay loop itself from conflict resolution overhead. In the table below, *Applied* denotes commands resolved as Committed or Duplicate — i.e. those that did not appear in the rejection manifest.

Commands	Time (ms)	Throughput (cmd/s)	Applied
100	0.02	4,907,975	72
500	0.09	5,780,347	368
1,000	0.16	6,448,160	734
5,000	0.81	6,196,747	3,752
10,000	1.44	6,957,533	7,418
50,000	9.36	5,340,668	37,018

Throughput is consistent at approximately 5–7 million commands per second across all log sizes, confirming that replay scales linearly with the number of commands (Figure 2). A log of 10,000 commands — large by the standards of a single offline session — replays in under 2 milliseconds, well below any perceptible threshold. The applied count is lower than the total because 30 % of commands are commutative and increment the entity version, causing subsequent non-commutative commands targeting the same entity to detect a version mismatch. This reflects realistic intra-session behaviour when a player modifies the same entity more than once.

During development, this benchmark identified a quadratic performance issue in the initial implementation. The original `ReplayAll` called `MarkFlushed` once per resolved command, each of which performed a linear scan of the pending vector. This produced $O(n^2)$ behaviour: 50,000 commands took 2939 ms before

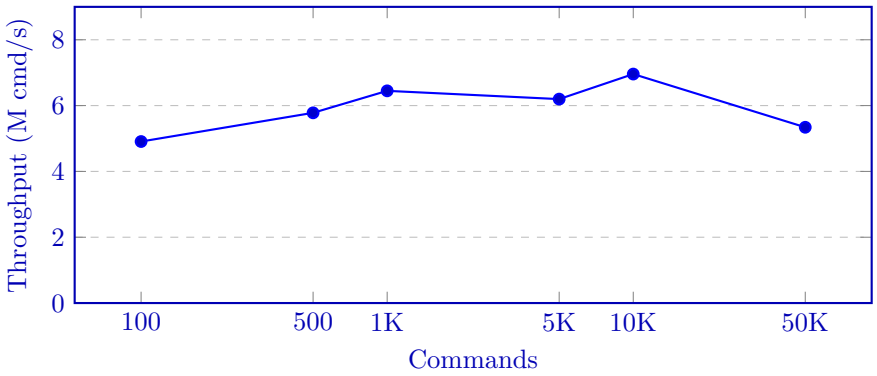

Figure 2: Replay throughput remains stable at 5–7 M cmd/s regardless of log size.

the fix. The fix — collecting all resolved keys and removing them in a single pass — reduced that to 9 ms, a 313-fold improvement.

7.3 Command Log Backend Comparison

The second benchmark compares the in-memory `CommandLog` against the SQLite-backed `SQLiteCommandLog` for append and retrieval operations.

Commands	Mem. Append (ms)	SQL Append (ms)	Mem. Get (ms)	SQL Get (ms)
100	0.00	0.33	0.00	0.03
1,000	0.01	3.35	0.00	0.20
5,000	0.06	19.96	0.02	1.06
10,000	0.13	33.57	0.03	1.82

SQLite append is approximately 250 times slower than in-memory (33 ms vs 0,13 ms for 10,000 commands), as shown in Figure 3; this is expected given that SQLite writes to disk with ACID guarantees. In absolute terms the overhead is still acceptable for an offline session: a player generating 10,000 actions — far more than a typical session — would incur roughly 33 ms of storage overhead. Retrieval via `GetPending` is fast in both backends; at 10,000 commands SQLite takes under 2 ms.

7.4 Conflict Detection Overhead

The third benchmark holds the command count fixed at 10,000 and varies the proportion of commands that are guaranteed to produce a version conflict, from 0 % to 100 %.

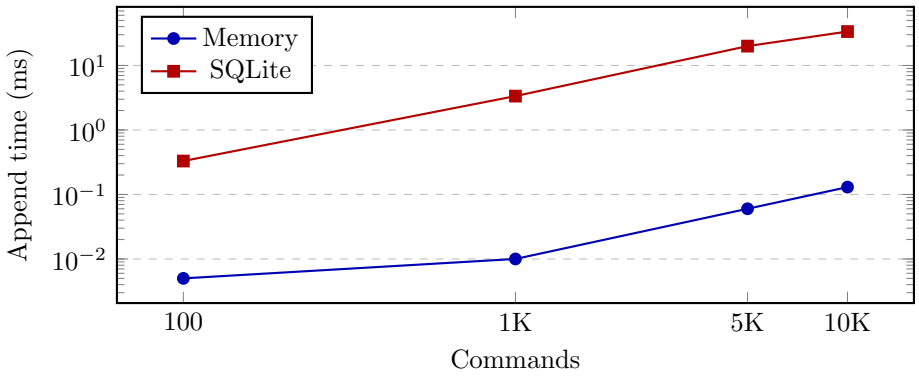

Figure 3: SQLite append is ∼250× slower than in-memory due to ACID disk writes; both scale linearly.

Conflict rate	Time (ms)	Throughput (cmd/s)	Applied
0%	1.10	9,130,686	3,246
25%	0.92	10,898,185	3,213
50%	0.93	10,696,617	3,175
75%	0.98	10,163,025	3,085
100%	0.89	11,230,176	2,912

Throughput varies by less than 20 % across the full range of conflict rates (Figure 4). Higher conflict rates are marginally faster because rejected commands skip the database write step. This confirms that the version check and classification logic add negligible overhead and that the system degrades gracefully under adversarial conflict conditions.

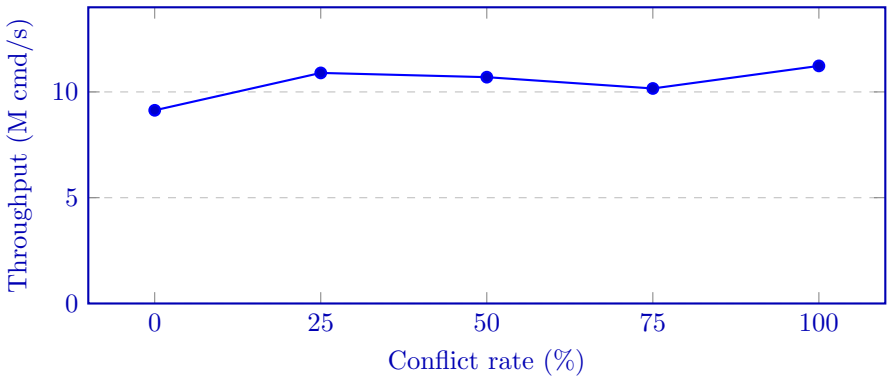

Figure 4: Throughput remains within 20 % across all conflict rates, confirming conflict detection adds negligible overhead.

7.5 Full Offline Session Round-Trip

The fourth benchmark measures the complete cost of an offline session using the SQLite command log: appending commands to the durable log, retrieving them, and replaying against the in-memory database.

Commands	Append (ms)	GetPending (ms)	ReplayAll (ms)	Total (ms)
100	0.30	0.02	0.15	0.47
1,000	2.86	0.17	0.81	3.84
5,000	15.26	0.84	3.09	19.19
10,000	30.42	1.70	6.29	38.40

A full session of 10,000 commands completes in approximately 38 ms end-to-end (Figure 5). SQLite append accounts for roughly 80 % of that time; actual replay takes only 6 ms. All three phases scale linearly with command count. The results indicate that reconnection latency attributable to synchronization is unlikely to be noticeable in practice for sessions of realistic size.

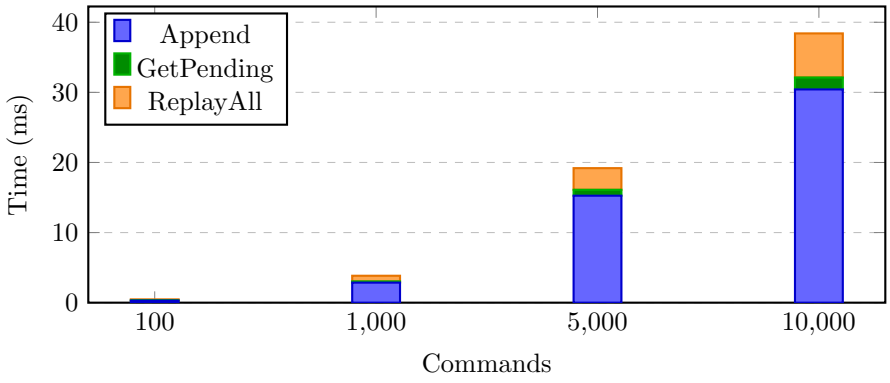

Figure 5: Full offline session cost broken down by phase; SQLite append dominates at ∼80 % of total time.

7.6 Command Log Disk Footprint

The fifth benchmark measures how much disk space the SQLite-backed command log consumes as a function of log size. Commands are generated with UUID-length idempotency keys (36 characters) to reflect realistic production identifiers. File size is measured after the database connection is closed, including any WAL file that was not checkpointed.

Commands	File size (KB)	Bytes per command
100	32.0	327.7
500	80.0	163.8
1,000	140.0	143.4
5,000	628.0	128.6
10,000	1,248.0	127.8

The per-command cost stabilizes at approximately 128 bytes for logs of 1000 commands or more. This overhead has two components: the raw data content of each row (idempotency key at 36 bytes, type string at approximately 13 bytes, entity ID, two integers, and timestamp — roughly 75 bytes in total) and SQLite's structural overhead from the B-tree page layout and the secondary index on the `idempotency_key` column, which adds approximately 50 bytes per row.

The high per-command cost at small log sizes (327 bytes at 100 commands) is a consequence of SQLite's fixed page size of 4 KB: even a single-row database occupies at least two pages — one for the data table and one for the unique index. Once the log grows large enough to fill pages efficiently the overhead drops to the steady-state figure.

In practical terms, a 1.000-command offline session — already a large session by most standards — occupies 140 KB. A session of 10.000 commands occupies approximately 1.2 MB (Figure 6). These figures are negligible on any modern storage medium and confirm that disk space is not a limiting factor for the command log approach.

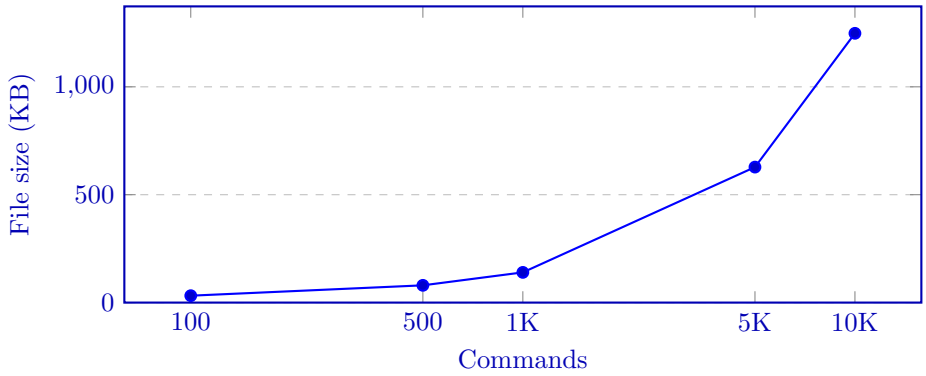

Figure 6: Command log file size grows linearly with command count; 10,000 commands occupy ∼1.2 MB.

7.7 Network Latency Model

All previous benchmarks measure local processing cost in isolation. To place those figures in context, the sixth experiment models the full reconnection time

by combining the measured local cost with a simple analytical network overhead. The command log is treated as a single batch request: the client serializes all pending commands, transmits them to the server, and waits for the rejection manifest. Under this model the network contribution to reconnection time is:

$$t_{\text{network}} = \frac{\text{RTT}}{1 - p_{\text{loss}}}$$

where RTT is the round-trip time and p_{loss} is the packet loss rate. The denominator accounts for geometric retransmission: on a lossy link each attempt succeeds with probability $1 - p_{\text{loss}}$, so the expected number of attempts is $1/(1 - p_{\text{loss}})$.

Five representative network conditions are tested against three command log sizes.

Network	Commands	Local (ms)	Network (ms)	Total (ms)
No network	100	0.52	0.00	0.52
LAN	100	0.52	2.00	2.52
Broadband	100	0.52	40.20	40.73
Mobile	100	0.52	122.45	122.97
Poor mobile	100	0.52	315.79	316.31
No network	1,000	4.12	0.00	4.12
LAN	1,000	4.12	2.00	6.12
Broadband	1,000	4.12	40.20	44.32
Mobile	1,000	4.12	122.45	126.57
Poor mobile	1,000	4.12	315.79	319.91
No network	10,000	40.22	0.00	40.22
LAN	10,000	40.22	2.00	42.22
Broadband	10,000	40.22	40.20	80.43
Mobile	10,000	40.22	122.45	162.67
Poor mobile	10,000	40.22	315.79	356.01

For small logs (100 commands), the local cost is negligible at 0,5 ms; reconnection time is dominated almost entirely by network latency regardless of log size. Even on a poor mobile connection (300 ms RTT, 5% loss), a 10,000-command log synchronizes in approximately 356 ms — well under half a second for what is already an unrealistically large offline session. Figure 7 visualises the local–network breakdown for the 10,000-command case.

The crossover point — where network overhead equals local processing cost — occurs at approximately 40 ms RTT, which corresponds to a mid-range broadband connection. Below that threshold (LAN), the synchronization mechanism itself is the bottleneck; above it, the network is. This suggests that for real-world deployments the primary lever for reducing reconnection time is reducing the size of the command log (for example, by coalescing commutative deltas) rather than optimising the local replay engine further.

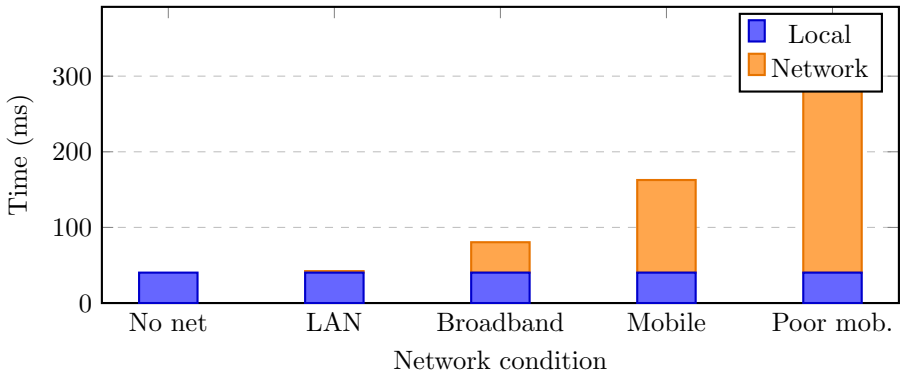

Figure 7: Reconnection time breakdown for 10,000 commands: local processing is constant at ∼40 ms; network RTT dominates under high-latency conditions.

8 Discussion

8.1 Architectural Trade-offs

Persistent world state in online multiplayer games can be implemented in many different ways. Each model has its own advantages and disadvantages in terms of consistency, flexibility, and implementation complexity.

Offline play becomes impossible in a purely server-authoritative architecture. However, the server-authoritative model does provide strong consistency by being a single source of truth. A database can't be consistent, offline and available at the same time due to the consequences of the CAP theorem (Gilbert & Lynch, 2002): consistency, availability, and partition tolerance cannot all be accomplished simultaneously, only two of the three are possible at a time. A server-authoritative model focuses on consistency, and will not accept modification to the database if the network is partitioned (i.e. the client is offline). In the hybrid approach, as presented in this thesis, writing to a local database is made possible while offline (during a partition), and is then later validated on the cloud-based server during reconnection. Consistency is sacrificed for the sake of availability, conflict resolution is used to restore consistency later on.

This hybrid approach is a middle-ground between an always online model, and an always offline model. The suggested Command-based model enables offline sessions, known as partitioning. Partitioned sessions can later be synchronized with the online server and allows a player to return online. Synchronization does not happen by taking the diffs of two databases and adding the diffs together: the suggested approach involves replaying commands recorded during offline play and replaying them on the server in the order in which they are recorded. The intent of the player, not only the changed state, is preserved. If the game has already been implemented with a server-authoritative architecture, transactions on the server stay the same. The design doesn't require a major

redesign. The most important aspect of this approach is to develop systems to recognize conflicts and to validate them on the server. Despite a complicated conflict-resolution implementation, this Command-based persistence method is still less invasive to the existing persistence layer in a server-authoritative architecture compared to adopting Conflict-free Replicated Data Types (CRDT) or the Operational Transformation (OT) approach.

8.2 Evaluation Findings

A C++ implementation was developed and benchmark results were made to support the architectural claims in the following ways:

The replay throughput experiments demonstrate that processing a command log of 10,000 commands takes less than 2 ms in-memory and 38 ms with SQLite. These values indicate that in practice logging and running synchronization on reconnection will likely result in negligible loading times. The data sent at reconnection with the authoritative server is proportional to the number of actions performed offline. It does not correlate with the size of the game world itself. In a large persistent world, this will likely result in less data being sent to the servers on reconnection compared to snapshot-based methods.

The conflict rate experiment shows something very useful: throughput varies by less than 20 % across the full range of conflict rates, from 0 % to 100 %. This means the system does not degrade meaningfully under adversarial conditions, and conflict detection can be considered a negligible overhead rather than a potential bottleneck.

Even on a bad mobile network (300 ms RTT, 5 % packet loss), the network latency model indicates that a 10,000-command log takes about 356 ms end-to-end to reconnect. If players have a fast connection, for example with latency below 40 ms RTT, then the server-side processing becomes the bottleneck. With a slower connection with latency greater than 40 ms RTT, actually sending the commands to the server is the bottleneck. This shows that making the commandlog smaller by optimization is the best way to speed up reconnection with the server for players with fast internet connections. One could for example remove commutative commands where possible, by merging them together.

During development, the benchmarking experiments revealed a major design flaw in the C++ implementation. The original `ReplayAll` called `MarkFlushed` per Command. This triggered a linear search through the command queue. This was slow and produced $O(n^2)$ behaviour. Code review alone was not enough to discover this mistake: 50,000 commands took nearly 3 seconds process. After the fix, processing 50,000 commands took only 9 ms. This shows how important it is to do test-driven development, and to avoid adding tests and doing testing ad-hoc late in development.

8.3 Scalability Considerations

The benchmarks are designed to show the performance of one client connected to the server at a time, which is not realistic on real-world servers. The benchmarks do not prove that the commandlog based implementation will scale well for thousands of players in a large-scale multiplayer game. There are many factors which go outside the scope of the current prototype which would affect the performance in a real-world environment.

In the prototype, the server-side validation is done against an in-memory store. This is a great oversimplification. The database is used to store entity versions and preconditions, but should really use a real SQL database. To properly test the system, many concurrent users should add load to the system, not only one user as in the benchmark. Validation throughput would depend heavily on features of the database implementation. A SQL database is typically designed to operate under heavy load. The benchmarks do not test what would happen on real databases under a heavy workload. This is hard to test with synthetically. Real player behaviour is unpredictable. They could connect and reconnect in a more stochastic random manner. Connections could be occur suddenly at certain times of the day. This would change the conflict rates measurement and could cause behaviour which was not caught by the tests and experiments conducted for this thesis.

However, despite these limitations, the core design of the commandlog offline-online synchronization model is still sound. But the benchmarks should not be used as proof of how the algorithms or model would perform in a real-world system.

8.4 Limitations and Threats to Validity

The conflict resolution in this thesis categorises all commands as three types: idempotent, commutative, and non-commutative. For a real-world system, this might not be enough. In a modern survival video game, the game world would encounter command types that go beyond the scope of the conflict resolution strategy found in this thesis. In particular, solving what should happen if a player loses items or buildings as a result of `ReplayCommand` rejecting a command, for instance because the entity is out of date on the client or has been removed on the online server-side, has not been properly discussed. The user should at the very least be notified, but it is up to the specific game implementation to figure out how to present and offer players a way to recover lost progress. Resolving these failures do not have a general solution and will need to make sense within the game's design.

The evaluation metrics are based on data generated synthetically by uniform random distribution. This does not represent a real offline session in regards to manipulation of entities in an actual game world and the command types in the tests are not of a typical distribution found in a game played by actual human-beings. The distribution of commands and entities will likely differ

based on game genre, session length, player experience, etc. It would have improved the thesis if actual telemetry from a real game was collected, future work could calibrate the tests and confirm if the system works under more realistic conditions.

8.5 Game Preservation

Beyond the technical design, offline functionality has an important cultural significance. Games are now considered important cultural artefacts of value. If a game depends on an online service exclusively, and the developer shuts down the servers, whether due to bankruptcy, acquisition, or commercial decision, the games made with this model risk becoming unplayable abandonware. These games might never become playable again without server-support. That's why adding offline functionality to a game means more than simply giving players the freedom to play online. It will also preserve the game for future generations. With offline, the game becomes playable independently of the publisher's infrastructure, this is currently relevant and discussions around game preservation are currently being made at the EU Parliament (Ondruška et al., 2026).

8.6 Financial Considerations

Offline capabilities also has practical consequences for both players and developers that extend beyond technical design.

For players, playing online often requires an online subscription. PlayStation Plus, Xbox Game Pass Core, or Nintendo Switch Online all cost money, and are required for online play. Adding offline functionality to a game will get rid of such barriers, making the game available to a greater number of players (Baker, 2025). Many people do not play games online on console, so adding offline broadens the reachable audience without requiring any change to the online multiplayer experience for players who do subscribe to online services.

Offline functionality has also got a monetary dimension for the developers as well. Running a large number of servers is expensive. An always online game with thousands of players must always be ready, even when players are not connected. Players expect the servers to be ready for them when they connect online. A hybrid architecture allows players to play offline without adding load to the servers. Thus, servers can be scaled down to the actual demand rather than being ready for any player who bought the game.

9 Future Work

There are many things this thesis did not explore with the C++ prototype, leaving some questions unanswered. Future research could entail:

Telemetry-calibrated benchmarks. The current stochastic evaluation of synthetic data is not ideal. The commandlog has been generated using uniform random distributions. Collecting real telemetry from offline gameplay sessions could calibrate the results and give better information about the design's performance.

Production server-side scaling. The current benchmarks did not use a real SQL database when simulating replay on the server-side. A production system would use an advanced and well tuned SQL database which would also be under stress from many concurrent connections and players enjoying the game. To properly consider the performance impact of the commandlog, the benchmark should use a system more similar to an actual production system under load.

Recovery paths for rejected commands. The current design returns a manifest of all the rejected commands. However, a solution to resolving and allowing players to recover what they lose have not been made in this thesis. A production system would need a way to communicate to the player which commands failed, why, and offer compensation. This is a game design problem, and should be taken seriously per game.

Command dependency graphs. Some commands might rely on previous commands succeeding from the offline commandlog. For example, upgrading a building would rely on the building being successfully built in the first place. If the building is rejected by the server, the upgrade command would no longer make sense. This has not been properly evaluated and thought about in the current design. It would require a dependency graph and would result in more sensible rejection manifests.

10 Conclusion

This thesis investigated the following research question: How can an offline-online synchronization system for game persistence targeting survival multiplayer games be designed and evaluated with respect to consistency, conflict resolution, and reconnection performance?

While considering this problem, the starting point was to analyze the weaknesses and strengths of a typical server-authoritative persistence model. In such a model, state changes are all done as transactional SQL commands against a centralized cloud backend. This is a robust persistence model implementation, with consistency guarantees. It does not allow offline player progression, however. With a server-authoritative model, there is no offline or local multiplayer game modes. To enable offline play, it is therefore important to find a way to handle divergent world state.

Two offline-only models were discussed, and evaluated before arriving at the suggested command-based design. The simplest way to make offline functionality in a game with a server-authoritative persistence model is to simply mirror the online database with a local SQLite database. To do this one should provide a consistent programming interface for persistence, a Database Interface (DBI) which routes all persistence logic to the right place. The persistence interface does not by itself know how to reconcile divergent state which occur during an offline session, complicating reconnecting online after playing solo. Another model considered was a Object–Relational Mapping layer. This model would not solve reconnection to an online service either. The model was eventually rejected due to a fundamental mismatch: an ORM tracks object state, the commandlog design tracks intent.

The proposed solution is a command-based replication model. Rather than attempting to synchronize static snapshots of two databases on reconnection online, the commandlog system records player intent as serializable commands that can be replayed on the authoritative server. This system does not say what the changes should be, but instead why the changes should happen. The commandlog system solves all three parts of the research question as follows:

Consistency is maintained by leaving the cloud server as the single authoritative source of truth. Offline commands are not committed to the server before they have been sent and validated server-side. Since each Command can translate into a DBI call, the same transactions can be applied during reconnection as during an online session.

Conflict resolution is the most important part of this design and is handled by classifying objects into three types. Furthermore, when the player goes offline, entity versioning is used to detect divergent state. The three types of commands are idempotent commands, which can be applied safely without rejection, commutative operations such as resource collection or experience points gained which can be merged via simple summation, and non-commutative oper-

ations, such as building a building or moving a vehicle, which needs to be either accepted or rejected server-side. Commands that are rejected will be sent and communicated to the player in a rejection manifest.

Reconnection performance was evaluated by running tests on a single machine, including a network latency model. A log of 10,000 commands was replayed on a simulated server system in under 2 ms in memory, and under 40 ms end-to-end including a more realistic SQLite persistence model. When network overhead is added, even a poor mobile connection (300 ms RTT, 5 % packet loss) produces a total reconnection time of approximately 356 ms. Conflict detection adds less than 20 % overhead regardless of conflict rate, and the command log file remains under 1.2 MB at 10,000 commands. The results show that the design itself is unlikely to cause bottlenecks when players reconnect to an online server.

When implementing this design it is important to remember that the commands must be deterministic and replay-safe. Commands must be stored in the order they were generated during an offline session. The server-side validation logic must cover all command types to prevent exploitation. These constraints are manageable, and should not require major refactoring of a server-authoritative system, but great care must be taken to implement this correctly on a production system.

In conclusion, the results suggest it is possible to use a command-based synchronization model to provide an extensible and practical hybrid offline–online persistence system for survival video games. Offline-online synchronization can be achieved without fundamentally redesigning an existing server-authoritative model.

Glossary

ACID Atomicity, Consistency, Isolation, Durability — the four properties that guarantee database transactions are processed reliably even in the event of errors, power failures, or concurrent access.

Atomicity Atomicity is a fundamental database principle ensuring transactions are treated as single, indivisible "all-or-nothing" units.

B-tree A self-balancing tree data structure that keeps data sorted and allows searches, insertions, and deletions in logarithmic time; the primary index structure used internally by SQLite.

Battle.net Blizzard Entertainment's online gaming platform and authentication infrastructure, required for connectivity in all modern Blizzard titles.

Catch2 A header-based C++ unit testing framework that provides test case macros, assertions, and test discovery without requiring a separate test runner.

CMake A cross-platform build system generator that produces native build files (Makefiles, Xcode projects, Visual Studio solutions) from a platform-independent description.

command log A durable, ordered sequence of command objects representing player actions recorded during an offline session; the primary artifact used to synchronize client state with the authoritative server on reconnection.

Command Pattern A software design pattern in which each action or request is encapsulated as a self-contained object, enabling serialization, queuing, logging, and replay of operations.

commutative operation An operation whose result is independent of the order in which it is applied relative to other operations; in this thesis, additive operations such as resource and reputation deltas that can be merged by summation.

CRDT Conflict-free Replicated Data Type — a mathematical data structure designed so that any two replicas can be merged in any order and will converge to the same result, guaranteeing eventual consistency without coordination.

DBI Database Interface — the abstract layer between game code and the underlying persistence backend; all state-modifying game actions are routed through the DBI, which can be swapped at runtime to target different backends.

Design Science Research A research methodology in which the primary output is an artifact — such as an architectural design or a working software prototype — rather than an empirical study; introduced by Hevner et al. (2004).

entity versioning A mechanism that assigns each persistent entity a monotonically increasing version number, incremented on every committed state change; used to detect whether an entity was modified between the time a command was generated and the time it is replayed.

event sourcing A software architecture pattern in which all changes to application state are stored as an append-only sequence of events, allowing the current state to be reconstructed by replaying the event log from the beginning.

idempotency The property of an operation whereby applying it multiple times produces the same result as applying it once; essential for safe retry logic and duplicate detection in distributed systems.

idempotency key A unique identifier (typically a UUID) attached to each command; the server uses it to detect and silently skip commands that have already been applied, enabling safe re-transmission after network failures.

listen server A listen server is a multiplayer game architecture where one player's computer acts as both the server and a client simultaneously. It allows for easy, cost-effective, local hosting without dedicated hardware, but the host's machine requires strong performance to handle both roles, and the session ends if the host leaves..

Monotonically Monotonically describes a process, function, or sequence that changes in only one direction—consistently increasing or consistently decreasing without reversing direction.

Mutex lock In computer science, a lock or mutex (from mutual exclusion) is a synchronization primitive that prevents state from being modified or accessed by multiple threads of execution at once.

non-commutative operation An operation whose result depends on the order in which it is applied; in this thesis, destructive or positional operations such as demolishing a building or placing a structure that must be validated against current server state before being accepted.

OCC Optimistic Concurrency Control — a concurrency strategy in which transactions proceed without acquiring locks, but validate that their preconditions still hold at commit time; conflicts cause the transaction to be rejected rather than blocked.

Operational Transformation A technique for reconciling concurrent edits by transforming operations so that they can be applied in any order while producing a consistent result; the mechanism underlying collaborative editing tools such as Google Docs.

ORM Object-Relational Mapping — a layer that translates between in-memory objects and relational database rows, allowing persistent entities to be stored and retrieved without writing explicit SQL.

rejection manifest A data structure returned to the client after a replay pass, listing each command that could not be applied along with the reason for rejection and the current server-side state of the affected entity.

replay The process of re-executing a sequence of commands against an authoritative data store in order to synchronize state; in this thesis, the mechanism by which offline commands are submitted to the server on reconnection.

server-authoritative architecture A system design in which the server is the single source of truth for all persistent state; all state-modifying actions are validated and committed server-side, and the client cannot unilaterally alter authoritative data.

Service Locator A software design pattern that provides a centralized registry through which components can obtain references to services (such as a database interface) at runtime, without being coupled to specific implementations.

SQLite An embedded, serverless, ACID-compliant relational database engine implemented as a C library; it stores an entire database in a single file on disk and runs within the host process, requiring no separate server.

Steam Cloud Save Valve's file-level save synchronization service, which uploads and downloads save files when a player connects; conflicts are resolved by last-write-wins, discarding any changes made on the other device.

Telemetry Telemetry is the automated collection and transmission of data and measurements from distributed or remote sources to a central system for monitoring, analysis and resource optimization.

UUID Universally Unique Identifier — a 128-bit identifier standardized in RFC 4122, typically represented as a 36-character hyphen-separated hexadecimal string; used as idempotency keys to uniquely identify each command.

WAL Write-Ahead Logging — a SQLite journal mode in which changes are written to a separate log file before being applied to the main database, improving concurrent read performance and crash recovery.

References

Baker, N. (2025). *Online gaming statistics*. https://www.uswitch.com/broadband/studies/online-gaming-statistics/

Bernstein, P. A., & Goodman, N. (1981). Concurrency control in distributed database systems. *ACM Computing Surveys*, *13*(2), 185–221. https://doi.org/10.1145/356842.356846

Blizzard Entertainment. (2004). *World of warcraft*. https://worldofwarcraft.blizzard.com

Blizzard Entertainment. (2012). *Diablo III*. https://diablo3.blizzard.com

Céré, J., Montiglio, P.-O., & Kelly, C. D. (2021). Indirect effect of familiarity on survival: A path analysis on video game data. *Animal Behaviour*, *181*, 105–116. https://doi.org/https://doi.org/10.1016/j.anbehav.2021.06.010

Ellis, C. A., & Gibbs, S. J. (1989). Concurrency control in groupware systems. *ACM SIGMOD Record*, *18*(2), 399–407. https://doi.org/10.1145/66926.66963

Fowler, M. (2005). *Event sourcing*. https://martinfowler.com/eaaDev/EventSourcing.html

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Gilbert, S., & Lynch, N. (2002). Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, *33*(2), 51–59. https://doi.org/10.1145/564585.564601

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, *28*(1), 75–105. https://doi.org/10.2307/25148625

Hořeňovský, M., & catchorg contributors. (2023). *Catch2*. https://github.com/catchorg/Catch2

Iron Gate Studio. (2021). *Valheim*. https://www.valheimgame.com

Kleppmann, M., Wiggins, A., van Hardenberg, P., & McGranaghan, M. (2019). Local-first software: You own your data, in spite of the cloud. *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*. https://doi.org/10.1145/3359591.3359737

Moll, P., Isak, S., Hellwagner, H., & Burke, J. (2021). A quadtree-based synchronization protocol for inter-server game state synchronization. *Computer Networks*, *185*, 107723. https://doi.org/https://doi.org/10.1016/j.comnet.2020.107723

Noland, M. (2014). Unreal property system (reflection). *Unreal Engine Blog*. https://www.unrealengine.com/blog/unreal-property-system-reflection

Nystrom, R. (2014a). *Game programming patterns*. Genever — Benning. https://books.google.no/books?id=9fIwBQAAQBAJ

Nystrom, R. (2014b). *Service locator — game programming patterns*. https://gameprogrammingpatterns.com/service-locator.html

Ondruška, D., Scott, R., Katzner, M.-M., Cerezo, A. H., & Barczyk, M. (2026). *Stop killing games at the european parliament*. https://www.youtube.com/watch?v=QXdmoeaYZ9Y

Shapiro, M., Preguiça, N., Baquero, C., & Zawirski, M. (2011). Conflict-free replicated data types, 386–400. https://doi.org/10.1007/978-3-642-24550-3_29

SQLite Consortium. (2024). *SQLite*. https://www.sqlite.org

Valve Corporation. (2023). *Steam cloud*. https://partner.steamgames.com/doc/features/cloud